

```
Sahil = ["Sahil", 123456, 23, 5.9, 80, True]
print(Sahil)
print(type(Sahil))
```

```
['Sahil', 123456, 23, 5.9, 80, True]
<class 'list'>
```

```
Sahil = ("Sahil", 123456, 23, 5.9, 80)
print(Sahil)
print(type(Sahil))
```

```
('Sahil', 123456, 23, 5.9, 80)
<class 'tuple'>
```

```
Sahil = {"Sahil", 123456, 23, 5.9, 80}
print(Sahil)
print(type(Sahil))
```

```
{123456, 80, 5.9, 'Sahil', 23}
<class 'set'>
```

PYTHON DATA STRUCTURES

Lists, Tuples, Sets

In real-world programming, we often need to store multiple values inside a single variable.

Imagine:

- 100 student names
- 1000 product prices
- 10,000 sensor readings

Creating separate variables is impossible.

That is why Python provides **Data Structures**.

They allow us to:

- Store multiple values
- Organize data
- Perform operations easily

✓ LIST

A List is:

- Ordered
- Changeable (Mutable)
- Allows duplicates
- Indexed

Lists are created using square brackets []

Used when:

- Data changes frequently
- Order matters
- Duplicates are allowed

Real-life examples:

- Student names
- Marks
- Shopping cart items

```
# Creating a list
students = ["Rahul", "Aman", "Neha", "Priya"]

print(students)
# a = "Sahil"
# print(a[0:4])
# Accessing elements (Indexing)
print(students[0]) # First element
print(students[-1]) # Last element

# Slicing
print(students[1:3])
```

```
['Rahul', 'Aman', 'Neha', 'Priya']
Rahul
Priya
['Aman', 'Neha']
```

```
numbers = [10, 20, 30]
print(numbers)
# Modify element
numbers[0] = 100
# numbers[3] = 100
print(numbers)

# Add element
numbers.append("Sahil")
print(numbers)

# Remove element
numbers.remove("Sahil")
print(numbers)
```

```
[10, 20, 30]
[100, 20, 30]
[100, 20, 30, 'Sahil']
[100, 20, 30]
```

```
A = [67,67,68,56,45]
A = [67,67,68,56,45]
A = [67,67,68,56,45]
A = [67,67,68,56,45]
A = [67,67,68,56,45]
```

```
marks = [85, 90, 78, 92, 48]
V = "65"
print("V")
average = sum(marks) / len(marks)
c = sum(marks)
print(c)
a = str((c / 500) * 100)
a = (c / 500) * 100
print(type(a))
# a = (sum(marks) / 500) * 100
b = "%"

# print(a + b)

print("Average Marks:", average)
print("marks")
```

```
V
393
<class 'float'>
Average Marks: 78.6
marks
```

✓ IN-DEPTH UNDERSTANDING OF LIST IN PYTHON

A List is:

- Ordered collection
- Mutable (changeable)
- Allows duplicates
- Indexed
- Can store multiple data types

Lists are the most used data structure in Python.

Used in:

- Data Science
- Machine Learning
- Web Development
- Competitive Programming

If you understand Lists deeply, you understand 60% of practical Python.

```
# Creating a list
numbers = [10, 20, 30, 40, 50]

print(numbers)
print(type(numbers))
```

```
[10, 20, 30, 40, 50]
<class 'list'>
```

✓ LIST PROPERTIES

1. Ordered → Maintains insertion order
2. Mutable → Can change elements
3. Allows duplicates
4. Indexed → Access via position
5. Can store mixed data types

```
mixed_list = ["Rahul", 21, 85.5, True]

print(mixed_list)
```

```
['Rahul', 21, 85.5, True]
```

```
students = ["Rahul", "Aman", "Neha", "Priya"]

# Positive indexing
print(students[0])
print(students[2])

# Negative indexing
print(students[-1])
print(students[-2])
```

```
Rahul
Neha
Priya
Neha
```

```
numbers = [10, 20, 30, 40, 50, 60]

print(numbers[1:4]) # 20, 30, 40
print(numbers[:3]) # first 3 elements
print(numbers[3:]) # from index 3 onward
print(numbers[::-2]) # step slicing
```

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
```

✓ LIST IS MUTABLE (VERY IMPORTANT)

Mutable means: You can modify, update, remove, and add elements.

This makes List powerful and flexible.

```
marks = [80, 85, 90]

# Modify element
marks[0] = 100
print(marks)

# Add element
marks.append(95)
print(marks)

# Insert at specific position
marks.insert(1, 88)
print(marks)

# Remove element
marks.remove(85)
print(marks)

# Remove using index
marks.pop()
print(marks)
```

```
[100, 85, 90]
[100, 85, 90, 95]
[100, 88, 85, 90, 95]
[100, 88, 90, 95]
[100, 88, 90]
```

✓ IMPORTANT LIST METHODS

- `append()` → Add at end
- `insert()` → Add at specific index
- `remove()` → Remove by value
- `pop()` → Remove by index
- `sort()` → Sort list
- `reverse()` → Reverse list
- `len()` → Length of list

```
numbers = [5, 2.98, 9, 1, 7]

numbers.sort()
print("Sorted:", numbers)

numbers.reverse()
print("Reversed:", numbers)

print("Length:", len(numbers))
```

```
Sorted: [1, 2.98, 5, 7, 9]
Reversed: [9, 7, 5, 2.98, 1]
Length: 5
```

✓ REAL LIFE EXAMPLE — STUDENT MARK ANALYSIS

Problem: You have marks of students. Find:

- Highest marks
- Lowest marks
- Average

```
marks = [85, 90, 78, 92, 88]

print("Highest:", max(marks))
print("Lowest:", min(marks))
print("Average:", sum(marks) / len(marks))
```

```
Highest: 92
Lowest: 78
Average: 86.6
```

✓ DUPLICATES IN LIST

Lists allow duplicates.

This is useful when:

- Recording transactions
- Storing scores
- Logging repeated data

```
scores = [85, 90, 85, 92, 90]

print(scores)
```

```
[85, 90, 85, 92, 90]
```

✓ NESTED LISTS (2D LIST)

List inside list.

Used for:

- Matrix
- Table data
- CSV rows

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[2][1])
a = [1, 2, 3]
# a[1][3]
a = [1, 2, 3]
b = [1, 2, 3]
c = [1, 2, 3]
mat = [a, b, c]
class_8 = [[12,13,14,15],[23,24,25,26],[19,17,13,15]]
print(mat)
print(mat[0])
# print(matrix[1][2])
```

```
3
[[1, 2, 3], [1, 2, 3], [1, 2, 3]]
[1, 2, 3]
```

```
mat = ['v', 'f', 'k']
print(mat)
print(mat[0])
```

```
['v', 'f', 'k']
v
```

✓ WHEN TO USE LIST?

Use List when:

- Data changes frequently
- Order matters
- Duplicates allowed
- You need indexing

Lists are dynamic and flexible.

```
# Mini challenge:  
# Take 5 numbers from user and store in list.  
# Print max, min, and average.
```

```
nums = []
```

```
nums.append(int(input("Enter number 1: ")))  
nums.append(int(input("Enter number 2: ")))  
nums.append(int(input("Enter number 3: ")))  
nums.append(int(input("Enter number 4: ")))  
nums.append(int(input("Enter number 5: ")))
```

```
print("Maximum:", max(nums))  
print("Minimum:", min(nums))  
print("Average:", sum(nums)/len(nums))
```

```
Enter number 1: 34  
Enter number 2: 23  
Enter number 3: 23  
Enter number 4: 23  
Enter number 5: 23  
Maximum: 34  
Minimum: 23  
Average: 25.2
```

✓ LIST PRACTICE QUESTIONS (VERY BASIC)

Answer in a new code cell.

1. Create a list named numbers containing: 10, 20, 30, 40.
2. Print the entire list.
3. Print the first element of the list.
4. Print the last element of the list using negative indexing.
5. Find and print the length of the list.
6. Add the number 50 to the end of the list.
7. Insert the number 15 at index position 1.
8. Change the value 30 to 300 in the list.
9. Remove the value 20 from the list.
10. Remove the last element using pop().
11. Create a list of 5 city names and print the third city.
12. Create a list with duplicate values (example: 1, 2, 2, 3, 3). Print the list.
13. Sort the list [5, 2, 9, 1] in ascending order.
14. Reverse the list [1, 2, 3, 4].
15. Create a list of marks (80, 85, 90, 95) and print:
 - Maximum marks
 - Minimum marks
 - Average marks

```
# =====
# SOLUTIONS - LIST BASIC PRACTICE
# =====

# 1
numbers = [10, 20, 30, 40]

# 2
print("Full list:", numbers)

# 3
print("First element:", numbers[0])

# 4
print("Last element:", numbers[-1])

# 5
print("Length of list:", len(numbers))

# 6
numbers.append(50)
print("After append 50:", numbers)

# 7
numbers.insert(1, 15)
print("After insert 15 at index 1:", numbers)

# 8
numbers[numbers.index(30)] = 300
print("After changing 30 to 300:", numbers)

# 9
numbers.remove(20)
print("After removing 20:", numbers)

# 10
numbers.pop()
print("After pop():", numbers)
```

```
# 11
cities = ["Delhi", "Mumbai", "Chennai", "Kolkata", "Bangalore"]
print("Third city:", cities[2])

# 12
duplicates = [1, 2, 2, 3, 3]
print("List with duplicates:", duplicates)

# 13
list1 = [5, 2, 9, 1]
list1.sort()
print("Sorted list:", list1)

# 14
list2 = [1, 2, 3, 4]
list2.reverse()
print("Reversed list:", list2)

# 15
marks = [80, 85, 90, 95]
print("Maximum marks:", max(marks))
print("Minimum marks:", min(marks))
print("Average marks:", sum(marks) / len(marks))
```

```
Full list: [10, 20, 30, 40]
First element: 10
Last element: 40
Length of list: 4
After append 50: [10, 20, 30, 40, 50]
After insert 15 at index 1: [10, 15, 20, 30, 40, 50]
After changing 30 to 300: [10, 15, 20, 300, 40, 50]
After removing 20: [10, 15, 300, 40, 50]
After pop(): [10, 15, 300, 40]
Third city: Chennai
List with duplicates: [1, 2, 2, 3, 3]
Sorted list: [1, 2, 5, 9]
Reversed list: [4, 3, 2, 1]
Maximum marks: 95
Minimum marks: 80
Average marks: 87.5
```

▼ TUPLE IN PYTHON

Tuple is:

- Ordered
- Immutable (cannot change after creation)
- Allows duplicates
- Indexed

Created using parentheses ()

Why Tuple Exists?

Because sometimes data should NOT change.

Example:

- Coordinates
- Date of birth
- Fixed configuration
- Database record snapshot

Tuples provide: ✓ Data safety ✓ Slightly better performance ✓ Can be used as dictionary keys

```
# Creating tuple
coordinates = (10.5, 20.3, 45)

print(coordinates)
print(type(coordinates))

# Indexing
print("First value:", coordinates[0])
```

```
# Slicing
print("Slice:", coordinates[0:2])
```

```
(10.5, 20.3, 45)
<class 'tuple'>
First value: 10.5
Slice: (10.5, 20.3)
```

```
# Mutability Demo
point = (5, 10)

# This will give error if uncommented
point[0] = 100

print("Tuple cannot be modified.")
```

```
-----
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_3855/709518308.py in <cell line: 0>()
      3
      4 # This will give error if uncommented
----> 5 point[0] = 100
      6
      7 print("Tuple cannot be modified.")

TypeError: 'tuple' object does not support item assignment
```

Next steps: [Explain error](#)

```
#tuple methods

numbers = (10, 20, 30, 20, 40)

# count(value)
# Returns number of times value appears

print("Count of 20:", numbers.count(20))

# index(value)
# Returns first index position of value

print("Index of 30:", numbers.index(30))
```

```
Count of 20: 2
Index of 30: 2
```

```
# Case 1 – GPS Coordinates (Should Not Change)

location = (28.7041, 77.1025) # Delhi coordinates
print("Latitude:", location[0])
```

```
Latitude: 28.7041
```

▼ SET IN PYTHON

Set is:

- Unordered
- Mutable
- No duplicates allowed
- Not indexed

Created using {}

Why Set Exists?

Because sometimes we only care about UNIQUE values.

Used in: ✓ Removing duplicates ✓ Membership testing ✓ Mathematical operations

```
numbers = {1, 2, 3, 3, 2}

print(numbers) # Duplicates removed automatically
```

```
{1, 2, 3}
```

```
A = {1, 2, 3}
B = {3, 4, 5}
C = {1,2,3,3,4,5}
# add(element)
A.add(6)
print("After add:", A)

# remove(element)
A.remove(2)
print("After remove:", A)

# discard(element) (no error if not found)
A.discard(100)

# pop() (removes random element)
A.pop()
print("After pop:", A)

# union(set)
print("Union:", A.union(B))

# intersection(set)
print("Intersection:", A.intersection(B))

# difference(set)
print("Difference B-A:", B.difference(A))
```

```
After add: {1, 2, 3, 6}
After remove: {1, 3, 6}
After pop: {3, 6}
Union: {3, 4, 5, 6}
Intersection: {3}
Difference B-A: {4, 5}
```

```
A = {1, 2, 3}
B = {3, 4, 5}

print("Union:", A.union(B))
```

```
Union: {1, 2, 3, 4, 5}
```

```
visitors = ["Rahul", "Aman", "Rahul", "Neha"]
unique_visitors = set(visitors)

print(unique_visitors)
```

```
{'Rahul', 'Aman', 'Neha'}
```

Type Conversion Between List, Tuple, and Set

Yes, conversion between list, tuple, and set is completely possible in Python.

This process is called:

Type Casting

or

Type Conversion

or more specifically

Collection Type Conversion

Python provides built-in constructor functions for this:

list() tuple() set()

These functions create a new object of the desired type from an existing collection.

Why Do We Convert Between Data Structures?

Because each structure behaves differently.

List:

- Ordered
- Mutable
- Allows duplicates
- Supports indexing

Tuple:

- Ordered
- Immutable
- Allows duplicates
- Supports indexing

Set:

- Unordered
- Mutable
- No duplicates
- No indexing

We convert when we need different behavior.

Examples:

- Remove duplicates → convert to set
- Make data fixed → convert to tuple
- Modify data → convert to list

```
# List to Tuple

my_list = [1, 2, 3, 4]

my_tuple = tuple(my_list)

print("Original List:", my_list)
print("Converted Tuple:", my_tuple)
print("Type:", type(my_tuple))
```

```
Original List: [1, 2, 3, 4]
Converted Tuple: (1, 2, 3, 4)
Type: <class 'tuple'>
```

List → Tuple

Why?

When you want to prevent modification. Once converted to tuple, the data becomes immutable.

Useful in:

- Configuration data
- Fixed records
- Coordinates

```
# List to Set

my_list = [1, 2, 2, 3, 3, 4]

my_set = set(my_list)

print("Original List:", my_list)
```

```
print("Converted Set:", my_set)
print("Type:", type(my_set))
```

```
Original List: [1, 2, 2, 3, 3, 4]
Converted Set: {1, 2, 3, 4}
Type: <class 'set'>
```

List → Set

Why?

To remove duplicates instantly.

Very common in:

- Data cleaning
- Removing repeated entries
- Membership checking

```
# Tuple to List
```

```
my_tuple = (10, 20, 30)
```

```
my_list = list(my_tuple)
```

```
print("Original Tuple:", my_tuple)
print("Converted List:", my_list)
print("Type:", type(my_list))
```

```
Original Tuple: (10, 20, 30)
Converted List: [10, 20, 30]
Type: <class 'list'>
```

Tuple → List

Why?

Tuple is immutable. If you want to modify elements, you must convert it into a list first.

```
# Tuple to Set
```

```
my_tuple = (1, 2, 2, 3)
```

```
my_set = set(my_tuple)
```

```
print("Original Tuple:", my_tuple)
print("Converted Set:", my_set)
print("Type:", type(my_set))
```

```
Original Tuple: (1, 2, 2, 3)
Converted Set: {1, 2, 3}
Type: <class 'set'>
```

Tuple → Set

Why?

To remove duplicates and perform set operations like union or intersection.

```
# Set to List
```

```
my_set = {5, 6, 7}
```

```
my_list = list(my_set)
```

```
print("Original Set:", my_set)
print("Converted List:", my_list)
print("Type:", type(my_list))
```

```
Original Set: {5, 6, 7}
Converted List: [5, 6, 7]
```

```
Type: <class 'list'>
```

Set → List

Why?

Set does not support indexing. If you need indexing, slicing, or sorting, convert it to a list.

```
# Set to Tuple

my_set = {8, 9, 10}

my_tuple = tuple(my_set)

print("Original Set:", my_set)
print("Converted Tuple:", my_tuple)
print("Type:", type(my_tuple))
```

```
Original Set: {8, 9, 10}
Converted Tuple: (8, 9, 10)
Type: <class 'tuple'>
```

Set → Tuple

Why?

To make unique data immutable. This is useful when you want fixed unique values.

✓ Important Technical Concept

When converting:

- `tuple(my_list)`
- `set(my_tuple)`
- `list(my_set)`

Python creates a NEW object in memory.

It does not modify the original object.

This is not in-place conversion. It is object reconstruction.

```
# Real-life example: Cleaning marks data

marks = [85, 90, 85, 92, 90]

# Remove duplicates
unique_marks = set(marks)

# Convert back to list for sorting
unique_marks = list(unique_marks)
unique_marks.sort()

print("Final Cleaned Marks:", unique_marks)
```

```
Final Cleaned Marks: [85, 90, 92]
```

Summary Table

✓ From To Function

List Tuple `tuple(list)` List Set `set(list)` Tuple List `list(tuple)` Tuple Set `set(tuple)` Set List `list(set)` Set Tuple `tuple(set)`

Final Understanding:

Convert when behavior needs to change.

Need uniqueness → set

Need immutability → tuple

Need modification or indexing → list

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

▼ TUPLE

A Tuple is:

- Ordered
- NOT changeable (Immutable)
- Allows duplicates
- Indexed

Created using parentheses ()

Used when:

- Data should NOT change
- Fixed configuration
- Safe data storage

Examples:

- GPS coordinates
- Date of birth

```
coordinates = (10.5, 20.3)

print(coordinates)
print(coordinates[0])

# Try modifying (will give error if uncommented)
# coordinates[0] = 99
```

```
(10.5, 20.3)
10.5
```

Why Tuple?

Immutability provides:

- Data safety
- Faster performance
- Protection from accidental changes

Tuples are often used for:

- Fixed values
- Returning multiple values from functions

▼ SET

A Set is:

- Unordered

- Changeable
- Does NOT allow duplicates
- NOT indexed

Created using curly braces {}

Used when:

- You only care about unique values
- Removing duplicates
- Membership checking

```
numbers = {1, 2, 3, 3, 2}

print(numbers) # Duplicates removed

# Add element
numbers.add(4)
print(numbers)

# Remove element
numbers.remove(2)
print(numbers)
```

```
{1, 2, 3}
{1, 2, 3, 4}
{1, 3, 4}
```

```
raw_scores = [85, 90, 85, 92, 90]

# Remove duplicates
unique_scores = set(raw_scores)
print(unique_scores)

# Convert back to list
unique_scores_list = list(unique_scores)
print(unique_scores_list)
```

```
{90, 92, 85}
[90, 92, 85]
```

COMPARISON: LIST vs TUPLE vs SET

Feature	List	Tuple	Set
Ordered	Yes	Yes	No
Changeable	Yes	No	Yes
Duplicates	Yes	Yes	No
Indexed	Yes	Yes	No

When to use:

- List → Flexible data
- Tuple → Fixed data
- Set → Unique data

Key Takeaway

Lists → Flexibility

Tuples → Safety

Sets → Uniqueness

Understanding these three properly builds your foundation for:

- Data Science
- Machine Learning